

Gestor de Recursos Distribuido HPC

R-322 — Sistemas Operativos I

Juan Ignacio Gorgo
Camila Sarasúa
Lucio Oviedo Dell'Orefice
Santiago Salvático

23 de junio de 2026

Índice

1. Descripción General del Proyecto	3
2. Arquitectura del Componente Erlang	3
2.1. Descripción General de los Módulos	3
2.2. Secuencia de Inicio del Agente C	5
2.3. Tolerancia a Fallos y Enlace de Procesos	6
2.4. Arquitectura de Concurrencia del Agente C	6
2.5. Motivación y Gestión de Conexiones Activas	6
2.6. Flujo de Procesamiento y Ciclo de Vida de los Mensajes	7
2.7. Contenedor Global del Nodo y Sincronización	8
2.8. Orquestación Centralizada, Escritura Asíncrona y Tolerancia a Fallos	9
3. Protocolo de Comunicación	10
3.1. Protocolo Inter-Agente (C a C)	10
3.2. Interfaz Local (Erlang a Agente C)	10
3.3. Diagrama de Secuencia: Ciclo de Vida de un Trabajo	11
4. Sincronización y Prevención de Deadlocks	12
4.1. Descripción del Problema	12
4.2. Estrategia: Ordenamiento Global de Recursos	12
4.3. Por Qué Funciona: Ejemplo Paso a Paso	12
4.4. Convención del Entorno de Pruebas	13
4.5. Análisis de Livelock	13
5. Registro de Eventos (Event Logger)	13
5.1. Diseño	13
5.2. Registro de Log (Log Record)	13
5.3. Ejemplo de Salida	14
5.4. Justificación del Diseño	14
6. Pruebas y Validación (test_deadlock.sh)	14

7. Estructuras de Datos	14
7.1. Nodos conocidos	15
7.2. Recursos locales	15
7.3. Cargas de trabajo propias	16
7.4. Sobre las implementaciones de los TADs empleados	17
8. Desafíos de Implementación y Soluciones	18
8.1. Identificación de Agentes	19
8.2. Propiedad de Recursos Remotos	19
8.3. Tráfico de Descubrimiento UDP	19
8.4. Colisión de Job ID entre Nodos en la Misma IP	19
8.5. Cierre Prematuro del Socket	20
8.6. Registro del ID del Proceso de Registro (Logger)	20
8.7. Bloqueo de Lectura TCP vs. Paso de Mensajes en Erlang	20
8.8. Fragmentación de Mensajes TCP (Framing)	20
9. Contribuciones del Equipo	21

1. Descripción General del Proyecto

Este informe describe el diseño e implementación de un gestor de recursos distribuido para un clúster HPC simulado. Cada nodo consta de dos componentes cooperativos: un **Agente Gestor de Recursos** escrito en C, y un **Planificador de Trabajos (Job Scheduler)** escrito en Erlang. Los componentes se comunican a través de TCP en localhost, mientras que los agentes C se comunican entre sí a través de la red utilizando un protocolo ASCII común.

El lado de Erlang es responsable de la generación de trabajos, la orquestación de solicitudes de recursos, la prevención de interbloqueos (deadlocks) y el registro de eventos. El lado de C gestiona los recursos locales, maneja conexiones concurrentes a través de `epoll` e implementa el descubrimiento de nodos basado en UDP.

El sistema consta de dos componentes principales por nodo: un agente C y un planificador Erlang. El lado de Erlang utiliza el modelo de actores: cada unidad lógica se ejecuta como un proceso independiente que se comunica exclusivamente a través del paso de mensajes, sin memoria compartida. El agente C maneja toda la E/S de red utilizando `epoll`, delegando la lógica de recursos a módulos internos conectados a través de una interfaz bien definida.

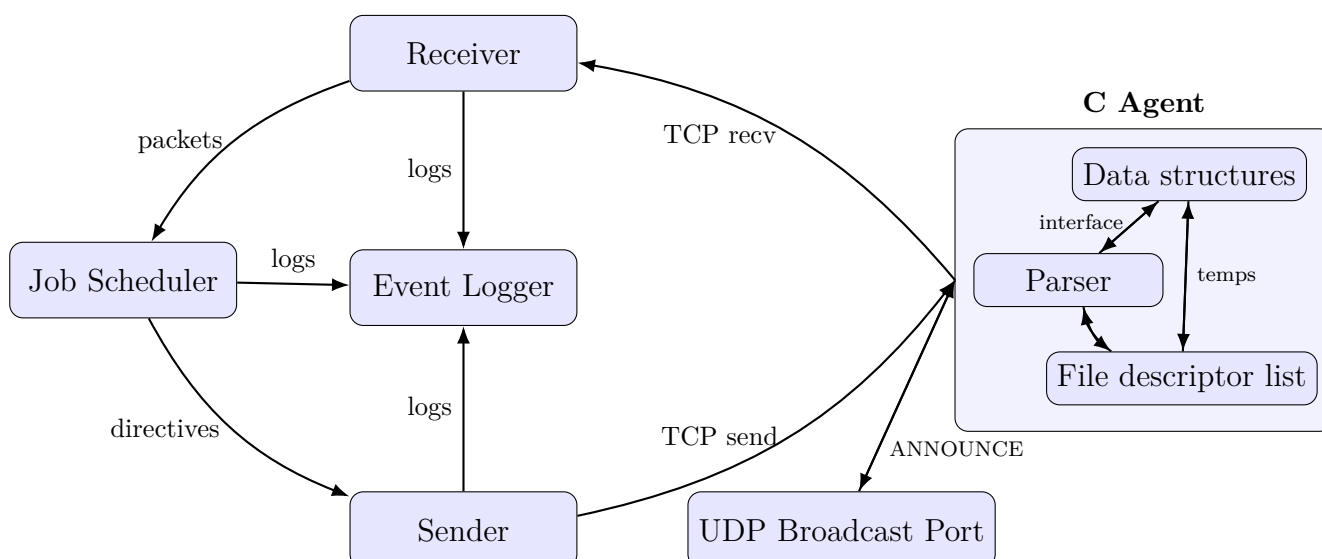


Figura 1: Simplificación del flujo de comunicaciones entre Erlang y C

2. Arquitectura del Componente Erlang

2.1. Descripción General de los Módulos

El componente Erlang está estructurado en cuatro módulos con responsabilidades claramente separadas:

Módulo	Responsabilidad
<code>header.hrl</code>	Macros compartidas, registros y constantes de ordenamiento de recursos
<code>erlang_c_bridge</code>	Gestión de conexiones TCP con el agente C local
<code>job_scheduler</code>	Generación de trabajos, lógica de planificación y gestión de estado
<code>event_logger</code>	Registro asíncrono de eventos en un archivo

Procesos livianos y el modelo de actores A diferencia de los hilos del sistema operativo utilizados en el agente C, los procesos de Erlang son entidades livianas administradas enteramente por la máquina virtual (BEAM), con un costo de creación y cambio de contexto mucho menor. Cada proceso posee su propio espacio de memoria privado: no existe memoria compartida entre procesos, por lo que toda comunicación ocurre exclusivamente mediante **paso de mensajes** asíncrono. Esto elimina por diseño la necesidad de *mutexes* o regiones críticas del lado de Erlang, ya que dos procesos nunca pueden modificar la misma estructura de datos al mismo tiempo: cada uno es dueño exclusivo de su propio estado.

Buzones de mensajes y receive selectivo. Cada proceso tiene asociado un buzón (*mailbox*) en el que se acumulan los mensajes enviados mediante el operador `!` (por ejemplo, `JobSchedulerId ! {packet_received, Packet}`). El proceso consume estos mensajes con la construcción `receive`, que realiza *pattern matching* sobre la forma del mensaje para decidir qué cláusula ejecutar:

```

1  receive
2      {receiver_pid, ReceiverId} -> ...;
3      {sender_pid, SenderId}    -> ...;
4      {packet_received, Packet} -> ...;
5      {error, Reason}           -> ...
6  end

```

Esta capacidad de discriminar mensajes por su estructura, en lugar de por el orden de llegada, es lo que permite que un mismo proceso (el `job_scheduler`) atienda simultáneamente eventos de naturaleza completamente distinta.

Recursión de cola como mecanismo de estado. Erlang no dispone de variables mutables ni de bucles `while/for` en el sentido imperativo: una vez asignada, una variable no puede reasignarse dentro del mismo ámbito. Por ello, procesos de larga duración como el `job_scheduler` implementan su ciclo de vida mediante **recursión de cola** (*tail recursion*): la función se llama a sí misma pasando el nuevo estado como argumento, en lugar de mutar una variable existente.

```

1  init(State, NRequests, Env) ->
2      receive
3          {job_granted, JobId} ->
4              State1 = maps:update(JobId, granted, State),
5              init(State1, NRequests, Env); % el "nuevo" estado se pasa
6          ...
7      end.

```

La BEAM optimiza estas llamadas para que no consuman espacio adicional en la pila, este patrón permite que un proceso viva de forma indefinida — procesando miles de mensajes — sin riesgo de desbordar la pila ni degradar el rendimiento.

Registros como estructuras de datos. Para representar entidades con múltiples campos (como un nodo del clúster o una entrada de log), se utilizan **registros** (*records*), definidos en `header.hrl`:

```

1  -record(node, {
2      ip,
3      port,
4      cpu,
5      mem,

```

```

6   gpu
7 }).
```

Los registros se traducen internamente a tuplas posicionales, pero permiten acceder y modificar campos por nombre (`Node#node.cpu`, `Node#node{cpu = 4}`), lo que mejora la legibilidad del código y reduce errores al reordenar o agregar campos.

Enlace de procesos como mecanismo de propagación de fallos. Tal como se detalla en la Sección 2.3, el mecanismo de `link/1` conecta bidireccionalmente el ciclo de vida de dos procesos: si uno termina de forma anómala (con una razón de salida distinta de `normal`), la señal se propaga automáticamente al otro, terminándolo también. Ante cualquier fallo interno de un proceso auxiliar, el sistema completo se detiene de forma visible en lugar de continuar en un estado inconsistente.

2.2. Secuencia de Inicio del Agente C

El agente C se inicia con seis argumentos: dirección IP, puerto TCP, los recursos disponibles de CPU, RAM y GPU, y el tamaño de la *thread pool*. La secuencia de inicio es la siguiente:

1. El `ServerContext` se inicializa con los argumentos proporcionados y se crea una nueva instancia de `epoll`.
2. Se abren tres sockets: un listener TCP público vinculado a la IP de la LAN del nodo, un listener TCP local vinculado exclusivamente a `127.0.0.1` para el planificador Erlang, y un socket UDP para el descubrimiento por difusión (broadcast).
3. Se configuran tres instancias de `timerfd`: un temporizador TCP de un solo disparo (2 segundos), un temporizador periódico de anuncios UDP (cada 1 segundo) y un temporizador periódico del recolector de basura (cada 5 segundos).
4. Los tres temporizadores y el socket UDP se registran en `epoll`. Los sockets TCP intencionalmente **no** se registran todavía.
5. Se inicializa un pool de hilos con 4 hilos de trabajo que comienzan a esperar tareas.
6. Comienza el bucle principal de `epoll`. Durante los primeros 2 segundos, solo se procesa tráfico UDP: el nodo difunde su presencia y recopila anuncios de pares que ya están en la red.
7. Cuando expira el temporizador TCP, ambos sockets TCP se añaden a `epoll` y el agente comienza a aceptar conexiones de otros agentes y del planificador Erlang local.

Del lado de Erlang, cuando se llama a `erlang_c_bridge:init/0`:

1. Genera (spawns) el proceso `event_logger` y lo registra bajo el átomo `eventLoggerId`.
2. Genera el proceso `job_scheduler`.
3. Llama a `connect/3`, que intenta hasta `?TRIES` conexiones TCP al agente C.
4. En caso de éxito, genera `conn_handler`, el cual genera a su vez `sender` y `receiver`, y notifica al planificador con sus PIDs.
5. Al agotarse los reintentos, registra el fallo en el log y llama a `erlang:halt/0`.

2.3. Tolerancia a Fallos y Enlace de Procesos

Para garantizar la tolerancia a fallos, y cumplir con el requisito de que, ante la caída de un proceso auxiliar (*worker*), el planificador completo también finalice, se utilizó el mecanismo de **enlace de procesos** (`link/1`) propio de Erlang. Cuando el planificador recibe los identificadores de proceso de `sender` y `receiver`, se enlaza a ambos:

```
1 {receiver_pid, ReceiverId} ->
2   link(ReceiverId),
3   ...
4 {sender_pid, SenderId} ->
5   link(SenderId),
6   ...
```

De esta manera, si cualquiera de estos dos procesos termina de forma anómala, la señal de salida se propaga al planificador y lo finaliza también, evitando que el sistema quede en un estado parcialmente vivo e inconsistente. El mismo criterio se aplicó al proceso que simula la ejecución de un trabajo (`simulate_load`), que se genera mediante `spawn_link/1` en lugar de `spawn/1`, de modo que una falla durante la simulación de carga también sea detectada por el planificador.

2.4. Arquitectura de Concurrencia del Agente C

El agente C se apoya en un único hilo principal que ejecuta el bucle de eventos de `epoll`, y un *pool* de hilos de trabajo (`thread_pool`) que procesa las tareas resultantes. El hilo principal nunca realiza operaciones bloqueantes: al detectar un evento en `epoll_wait`, lo clasifica y, según su tipo, lo atiende directamente (expiración de temporizadores, nuevas conexiones TCP) o lo encola como una `WorkerTask` para que un hilo del *pool* la procese de forma asíncrona.

Todos los descriptores de archivo correspondientes a conexiones TCP se registran en `epoll` con la bandera `EPOLLONESHOT`. Esta bandera indica al kernel que, tras notificar un evento sobre un descriptor, deje de vigilarlo hasta que se lo rearme explícitamente. Esto evita una condición de carrera real: sin `EPOLLONESHOT`, si llegaran nuevos datos a un socket mientras un hilo del *pool* todavía está procesando el evento anterior sobre ese mismo socket, `epoll` podría notificar el mismo descriptor a un segundo hilo, resultando en dos hilos leyendo el mismo socket simultáneamente. Al desarmar el descriptor tras cada evento, se garantiza que un socket solo puede estar siendo procesado por un hilo a la vez; una vez que el hilo termina, rearma el descriptor mediante `rearm_epoll_fd`, y `epoll` vuelve a vigilarlo con normalidad. Ningún dato se pierde durante el intervalo en que el descriptor está desarmado, ya que el kernel conserva los datos entrantes en el buffer del socket hasta que se lo vuelve a registrar.

La tabla `active_connections`, que asocia cada descriptor de archivo con su IP, puerto y estado, se protege con un `pthread_rwlock`. Dado que la mayoría de los accesos a esta tabla son de lectura (consultar si existe una conexión hacia determinado nodo) y las escrituras son comparativamente poco frecuentes (alta o baja de una conexión), este tipo de lock permite que múltiples hilos lectores accedan concurrentemente, serializando únicamente las operaciones de escritura.

2.5. Motivación y Gestión de Conexiones Activas

La tabla `active_connections` centraliza el estado de todos los sockets TCP del agente, pero su diseño y comportamiento están estrictamente dictados por la dualidad del sistema, el cual debe operar simultáneamente como cliente y como servidor.

Separación de Módulos y Locks Independientes. Esta necesidad de manejar tráfico bidireccional asíncrono motivó la separación de la lógica del agente en distintos archivos C, aislando el estado del servidor (`local_resources`) del estado del cliente (`owned_jobs`) y del directorio general (`known_nodes`). Esta modularización estructural es precisamente lo que justifica la implementación de tres *locks* independientes (`lock_local`, `lock_owned` y `lock_known`) en lugar de un único *mutex* global. Gracias a esto, el sistema evita la contención cruzada: la recepción y encolado de una petición entrante no bloquea la generación simultánea de una nueva solicitud saliente, maximizando la concurrencia del *thread pool*.

Mensajes Entrantes (Rol Servidor). Cuando se trata de conexiones entrantes, el agente actúa en su rol de servidor (delegando la lógica a `local_resources`). En este escenario, el sistema se basa **exclusivamente en el descriptor de archivo (fd)** para identificar unívocamente la sesión.

Esto se debe a una limitación técnica fundamental: a nivel de socket, el sistema operativo receptor solo puede ver el puerto efímero (aleatorio) del remitente, imposibilitando descubrir de forma fiable cuál es el puerto público real donde ese nodo está escuchando anuncios. A su vez, basar la identificación únicamente en la dirección IP es inviable, ya que la arquitectura permite explícitamente que múltiples nodos se ejecuten sobre la misma máquina (misma IP) en distintos puertos. Por esta razón, en la faceta de servidor **no existe reutilización de descriptores** para tráfico cruzado; el FD es la única garantía de identidad.

Nota: El cliente Erlang local es tratado como un caso especial de conexión entrante. Su conexión sobre el puerto local (`<port>+ 1`) se registra en `active_connections` y se identifica exclusivamente por su FD, manteniéndose persistente y aislada del tráfico público.

Mensajes Salientes (Rol Cliente). Por el contrario, para la emisión de mensajes salientes, el agente asume el rol de cliente (gestionado por `owned_jobs`). Aquí el comportamiento frente a `active_connections` cambia radicalmente para optimizar el uso de la red: **sí existe reutilización de descriptores**.

Cuando el orquestador `resource_adapter.c` necesita enviar una petición (por ejemplo, un comando `RESERVE`) a un nodo remoto, el sistema sí conoce con exactitud la IP y el puerto de escucha destino, obtenidos a través del directorio `known_nodes`. Aprovechando esto, el orquestador invoca la función `search_fd_by_ip_port`. Esta función consulta la tabla de conexiones para verificar si ya existe un socket saliente activo hacia esa dupla (IP, Puerto). De ser así, se reutiliza el mismo FD para enrutar la nueva petición. Esta reutilización es crítica para el rendimiento del clúster, ya que evita la latencia y la sobrecarga que implicaría realizar un nuevo *handshake* TCP por cada recurso individual solicitado a un mismo destino.

2.6. Flujo de Procesamiento y Ciclo de Vida de los Mensajes

La arquitectura asíncrona del agente C delega el procesamiento de mensajes a través de una tubería clara. El ciclo de vida de un mensaje entrante comienza en el hilo principal y finaliza con la intervención de un módulo específico que determina la respuesta. El flujo es orquestado centralmente por `resource_adapter.c`, operando de la siguiente manera:

1. **Recepción (`epoll` y *Thread Pool*):** El hilo principal detecta un evento (TCP o UDP) y lo encola como una tarea asíncrona para que un hilo de trabajo (*worker*) extraiga los datos del socket.
2. **Orquestación y Enrutamiento (`resource_adapter.c`):** Este módulo actúa como el controlador principal. Parsea el mensaje entrante, determina el tipo de directiva, adquiere

el *lock* correspondiente (*lock_local*, *lock_known* o *lock_owned*) y deriva la ejecución al módulo responsable:

- **Descubrimiento UDP:** Ante un mensaje **ANNOUNCE**, se invoca al módulo **known_nodes**. Este actualiza la marca temporal en la tabla hash o inserta un nuevo nodo. No genera una respuesta de red.
- **Peticiones Entrantes (Rol Servidor):** Cuando se recibe un **RESERVE** por el puerto público, el mensaje se envía a **local_resources**. Este módulo evalúa la disponibilidad física; si hay recursos, emite directamente la respuesta **GRANTED** al nodo remoto. Si no, encola la petición (*request_queue*).
- **Directivas del Planificador (TCP Local):** Un **JOB_REQUEST** proveniente de Erlang es delegado a **owned_jobs**. El agente asume su Rol Cliente, inicializa las estructuras del trabajo y envía peticiones **RESERVE** a los nodos remotos implicados.
- **Respuestas de Nodos Remotos (Rol Cliente):** La recepción de un **GRANTED** o **DENIED** desde un agente externo también es manejada por **owned_jobs**. Se actualiza el estado de la reserva en vuelo y, si el trabajo ha obtenido la totalidad de sus recursos, este módulo se encarga de emitir la respuesta final **JOB_GRANTED** hacia el planificador Erlang local.

El siguiente diagrama ilustra cómo *epoll* alimenta el sistema y cómo *resource_adapter.c* divide los trabajos y desencadena las respuestas según el rol de cada módulo:

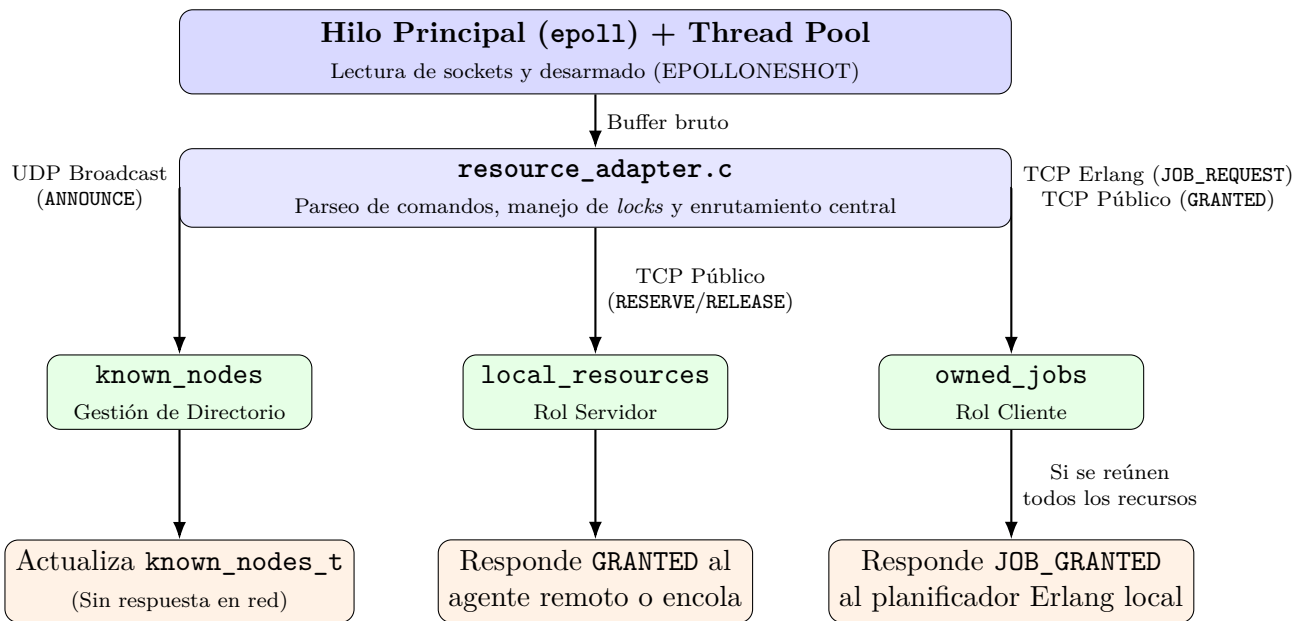


Figura 2: Ciclo de vida de los mensajes: desde su recepción en *epoll* hasta la respuesta de red, orquestado a través de *resource_adapter.c*.

2.7. Contenedor Global del Nodo y Sincronización

La información de cada nodo se unifica en una única estructura opaca, *node_data_t*, que actúa como contenedor de los tres módulos principales (*known_nodes*, *local_resources* y *owned_jobs*), descritos en detalle en la Sección 7. En lugar de utilizar un único *mutex* global para proteger todo el estado del nodo, se optó por un esquema de **tres locks independientes**, uno por módulo:

- **lock_local**: protege los recursos físicos propios y la tabla de trabajos activos concedidos a nodos remotos.
- **lock_known**: protege el directorio de nodos conocidos (descubiertos vía **ANNOUNCE**).
- **lock_owned**: protege la tabla de trabajos iniciados por el planificador Erlang local.

Esta separación reduce la contención entre operaciones que, en la práctica, no interfieren entre sí: por ejemplo, actualizar el directorio de nodos conocidos ante un **ANNOUNCE** no debería bloquear el procesamiento de una reserva local (**RESERVE**). Cuando una misma operación necesita interactuar con más de un módulo (por ejemplo, al detectar un nodo caído, que requiere leer **known_nodes**, identificar los trabajos afectados en **owned_jobs**, y liberar los recursos correspondientes en **local_resources**), se respeta un orden de adquisición y liberación consistente entre las distintas rutas del código para evitar la posibilidad de un interbloqueo entre hilos a nivel de los propios *locks* del agente C.

2.8. Orquestación Centralizada, Escritura Asíncrona y Tolerancia a Fallos

El módulo **resource_adapter.c** no solo actúa como puente entre la red y las estructuras de datos, sino que es el orquestador principal que garantiza la consistencia del agente C. Sus responsabilidades abarcan desde la ejecución segura de comandos hasta la gestión de escrituras no bloqueantes y la recuperación ante caídas.

Enrutamiento de Comandos (El Switch Central). Una vez que el hilo trabajador extrae y decodifica el comando del buffer entrante (fase de *parsing*), el adaptador de recursos recibe una estructura con la directiva parseada. Para procesarla, emplea una estructura de *switch-case* cuyo propósito fundamental es estandarizar la adquisición de los *locks* (**lock_local**, **lock_known** o **lock_owned**) antes de derivar la ejecución a los módulos correspondientes (**local_resources** u **owned_jobs**).

Esta separación estricta en el *switch* aísla la lógica de red de la lógica de negocio y asegura que el *thread pool* ejecute rutas de código predecibles. Al adquirir los locks en un orden predefinido según el tipo de comando (por ejemplo, peticiones de Erlang vs. tráfico cruzado entre agentes), se elimina la posibilidad de un interbloqueo interno a nivel de los hilos de C.

Gestión de Escrituras No Bloqueantes (Outbox). Dado que el agente emplea **epoll** y los descriptores de archivo están configurados como no bloqueantes (**O_NONBLOCK**), existe la posibilidad de que una llamada a sistema como **send()** o **write()** no pueda transmitir la totalidad del mensaje inmediatamente (retornando errores como **EAGAIN** o **EWouldBlock**) si los buffers del kernel están saturados. Bloquear el hilo trabajador a la espera de que la red se libere degradaría la arquitectura concurrente.

Para solucionar esto, el sistema asocia una cola de salida (*Outbox*) a cada conexión activa. Si el adaptador de recursos intenta enviar una respuesta y el kernel solo acepta una parte de los bytes, el agente encola el remanente en el *Outbox* del socket y rearma el descriptor en **epoll** añadiendo la bandera **EPOLLOUT**. Cuando el socket vuelve a estar disponible para escritura, **epoll** notifica al hilo principal, un trabajador toma la tarea y vacía progresivamente el *Outbox*. Esto garantiza que ningún mensaje se pierda o corrompa, independientemente de la congestión de la red.

Rutinas de Recuperación ante Fallos. En un entorno distribuido, las fallas son inevitables. `resource_adapter.c` centraliza el manejo de desconexiones en tres escenarios, garantizando que los recursos retenidos siempre se liberen:

1. **Caída del Planificador Erlang (TCP Local):** Si el proceso Erlang muere o el socket local se cierra abruptamente, el adaptador detecta el evento y purga el módulo `owned_jobs`. Esto implica cancelar localmente todos los trabajos en vuelo y emitir de forma proactiva mensajes `RELEASE` a los agentes remotos. El agente C no finaliza; regresa a su estado base esperando que un nuevo planificador se conecte.
2. **Caída de un Agente Remoto (TCP Público):** Si se corta la conexión directa con un agente, se dispara una doble rutina de limpieza orquestada por el adaptador. Primero, invoca a `local_resources` para liberar las reservas físicas que el nodo caído tenía en nuestra máquina. Segundo, invoca a `owned_jobs` para evaluar qué trabajos locales dependían de ese nodo remoto; al no poder completarse, se marcan como fallidos y se envía un `JOB_DENIED` al planificador Erlang.
3. **Nodo Inalcanzable (Timeout UDP):** Si el módulo `known_nodes` detecta que un nodo ha excedido su límite de tiempo sin enviar mensajes `ANNOUNCE`, se lo elimina del directorio. Aunque no haya un evento de cierre TCP explícito, esta eliminación programática fuerza al adaptador a desencadenar exactamente la misma doble rutina de limpieza descrita en el paso anterior, purgando el clúster de nodos "fantasma".

3. Protocolo de Comunicación

3.1. Protocolo Inter-Agente (C a C)

Los agentes C se comunican directamente entre sí a través de TCP utilizando un protocolo ASCII orientado a líneas, donde cada mensaje termina con `\n`. Este protocolo se utiliza para reservar y liberar recursos en nodos remotos.

Mensajes enviados entre agentes:

```
1 RESERVE <job_id> <resource_name> <amount>
2 GRANTED <job_id>
3 DENIED <job_id>
4 RELEASE <job_id> <resource_name> <amount>
```

Donde `job_id` es un entero único generado por el planificador Erlang, `resource_name` es uno de `cpu`, `mem` o `gpu`, y `amount` es la cantidad solicitada. El descubrimiento de nodos se maneja mediante difusión (broadcast) UDP. Cada agente anuncia periódicamente su presencia con:

```
1 ANNOUNCE <port> <cpu:N> <mem:N> <gpu:N>
```

La IP del remitente se extrae directamente de `recvfrom()` en lugar de incluirse en el mensaje. Si no se recibe ningún anuncio de un nodo dentro de los 15 segundos, se considera muerto y se elimina de la tabla de nodos conocidos.

3.2. Interfaz Local (Erlang a Agente C)

El proceso `sender` reenvía las directivas del planificador al agente C:

```
1 GET_NODES
2 JOB_REQUEST <job_id> @<ip>:<port>:<resource>:<amount> ...
```

```
3 JOB_RELEASE <job_id>
```

El proceso `receiver` escucha en el mismo socket y reenvía las respuestas al planificador:

```
1 NODES <ip>:<port>:<res>:<amount>:...;<ip>:...
2 JOB_GRANTED <job_id>
3 JOB_DENIED <job_id>
4 JOB_TIMEOUT <job_id>
```

3.3. Diagrama de Secuencia: Ciclo de Vida de un Trabajo

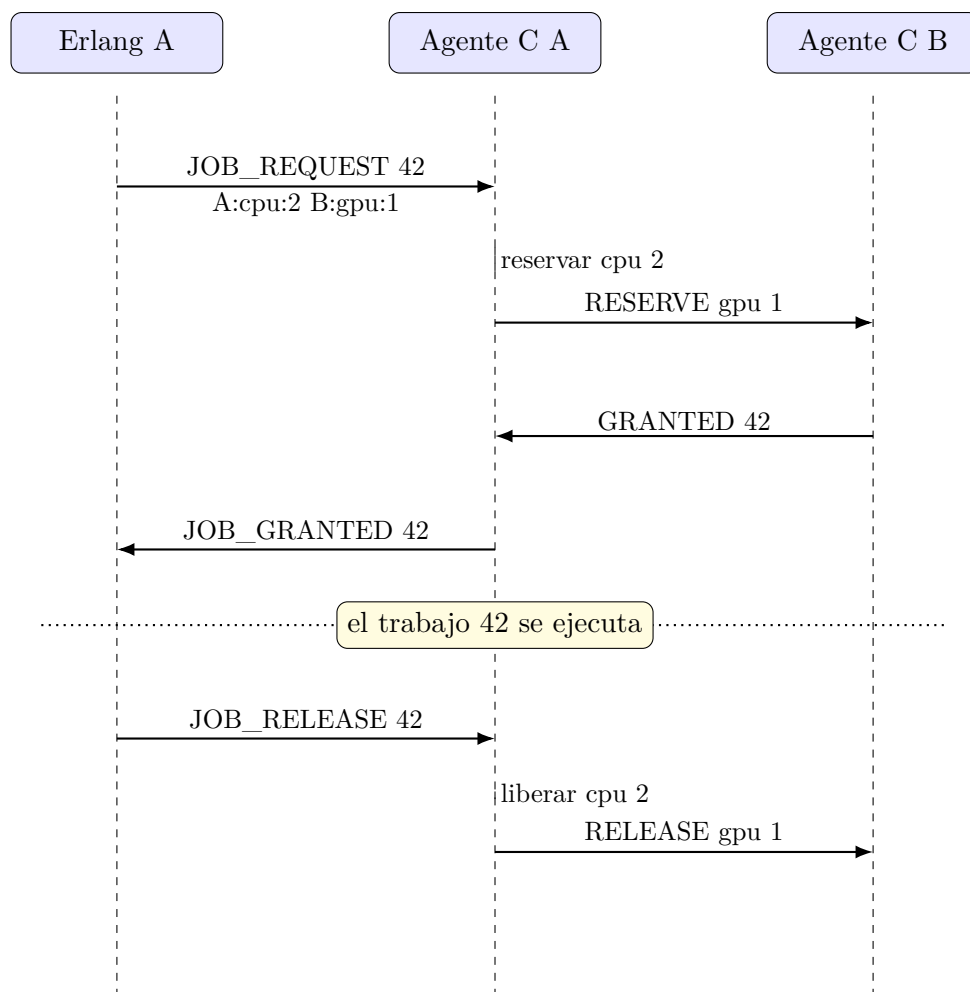


Figura 3: Ejecución y liberación exitosa de un trabajo.

4. Sincronización y Prevención de Deadlocks

4.1. Descripción del Problema

El deadlock distribuido ocurre cuando dos trabajos retienen cada uno un recurso que el otro necesita, formando una espera circular. El escenario clásico de la consigna:

- **Job1** (del Nodo A): necesita 2 CPUs del Nodo A y 1 GPU del Nodo B.
- **Job2** (del Nodo B): necesita 1 GPU del Nodo B y 2 CPUs del Nodo A.

Si ambos trabajos adquieren su primer recurso simultáneamente, cada uno se bloquea esperando el recurso retenido por el otro, lo que genera un deadlock.

4.2. Estrategia: Ordenamiento Global de Recursos

La estrategia elegida elimina la condición de **espera circular** (condición 4 de Coffman) al imponer un ordenamiento global en todas las solicitudes de recursos antes de enviarlas. Antes de enviar un `JOB_REQUEST`, el planificador ordena la lista de recursos requeridos por el par `(NodeIp, ResourceType)` en orden ascendente. El orden de los tipos se define de forma centralizada en `header.hrl`:

```
1 -define(RES_ORDER(Type),  
2     case Type of  
3         cpu -> 1;  
4         mem -> 2;  
5         gpu -> 3  
6     end  
7 ).
```

El ordenamiento se aplica en `generate_job_request/2`:

```
1 SortedResources = lists:sort(  
2     fun ({Ip1, Resource1, _}, {Ip2, Resource2, _}) ->  
3         {Ip1, Resource1} =< {Ip2, Resource2}  
4     end,  
5     AvailableResources  
6 ),
```

4.3. Por Qué Funciona: Ejemplo Paso a Paso

Con el invariante de ordenamiento aplicado, el escenario de deadlock se resuelve de la siguiente manera:

1. Tanto el Trabajo1 como el Trabajo2 ordenan sus listas de recursos. Dado que $A < B$ lexicográficamente, ambas listas comienzan con los recursos del Nodo A.
2. Ambos trabajos compiten primero por las CPUs del Nodo A. Uno gana; el otro espera en la cola FIFO del Nodo A.
3. El trabajo ganador (Trabajo1) luego solicita la GPU del Nodo B y la adquiere.
4. El Trabajo1 finaliza y libera ambos recursos. El Trabajo2 luego adquiere las CPUs del Nodo A, luego la GPU del Nodo B y se completa normalmente.

El invariante clave es: *ningún trabajo puede retener un recurso de un nodo “posterior” mientras espera un recurso de un nodo “anterior”*. La espera circular es estructuralmente imposible.

4.4. Convención del Entorno de Pruebas

Para simplificar las pruebas de interbloqueo sin las restricciones del ordenamiento global de recursos, se introdujo una convención para la variable de entorno `ENV`. Cuando `ENV` se establece en `"test"`, la función `generate_job_request` omite el paso de ordenamiento de recursos, permitiendo que los trabajos soliciten recursos en un orden arbitrario. Esto hace posible activar deliberadamente el escenario de interbloqueo descrito en la Sección 4 y verificar que el sistema lo detecte o se recupere de él.

En producción (`ENV` sin establecer o con cualquier otro valor), siempre se aplica el ordenamiento global y el interbloqueo se previene estructuralmente.

4.5. Análisis de Livelock

Además del interbloqueo, se analizó la posibilidad de un *livelock*: un escenario en el que dos trabajos compiten repetidamente por el mismo recurso, ambos son rechazados, reintentan, y vuelven a competir indefinidamente sin que ninguno progrese. En la implementación actual del agente C, cuando un recurso no está disponible, la solicitud no se rechaza con `DENIED`: se encola en una cola FIFO específica del recurso (`request_queue`) y se atiende automáticamente en cuanto el recurso se libera, respetando el orden de llegada. Esto garantiza que toda solicitud eventualmente progresa, sin necesidad de que el planificador Erlang reintente. Por lo tanto, bajo el diseño actual, el livelock no puede ocurrir: no existe un camino en el que dos trabajos queden reintentando la misma solicitud en bucle, ya que el agente C nunca deniega un recurso simplemente por no estar disponible en el momento de la solicitud, sino que garantiza su entrega ordenada mediante la cola de espera.

5. Registro de Eventos (Event Logger)

5.1. Diseño

El logger se ejecuta como un proceso Erlang dedicado, registrado bajo el átomo `eventLoggerId`. Cualquier módulo registra un evento llamando a:

```
1 event_logger:log_event(Status, SrcMethod, Detail, JobInvolved)
```

Esto construye un registro `#logInfo{}` y lo envía como un mensaje al logger, que lo escribe en `event_logger.txt` en modo de adición (append) utilizando I/O de archivos nativa (`raw`).

5.2. Registro de Log (Log Record)

```
1 -record(logInfo, {
2     status,           % ok | error | close
3     src_method,       % {Modulo, Funcion}
4     detail,           % atom o string que describe el evento
5     job_involved,     % entero job_id | none
6     timestamp         % {{aaaa,mm,dd},{hora,min,seg}}
7 }).
```

5.3. Ejemplo de Salida

```

1 ok {erlang_c_bridge,init} initialization none {14,32,10}
2 ok {erlang_c_bridge,connect} "CONNECTION SUCCESSFUL" none
   {{2006,5,11},{14,32,10}}
3 ok {job_scheduler,init} "[job_scheduler] sender_pid received" none
   {{2006,5,11},{14,32,11}}
4 ok {job_scheduler,init} "[job_scheduler] job_request" 5
   {{2006,5,11},{14,32,16}}
5 ok {job_scheduler,init} "[job_scheduler] job_granted" 5
   {{2006,5,11},{14,32,20}}

```

5.4. Justificación del Diseño

Registrar el logger bajo un átomo fijo evita tener que pasar el PID del logger a través de cada llamada a función. Cualquier proceso en el sistema puede registrar un evento en cualquier momento sin necesidad de mantener una referencia.

6. Pruebas y Validación (`test_deadlock.sh`)

Para cumplir con el requisito de demostrar que la implementación evita el escenario de interbloqueo descrito en la Sección 4, se provee el script `test_deadlock.sh`. Este script automatiza la ejecución de dos instancias del agente C en puertos distintos (simulando dos nodos del clúster en la misma máquina), y luego inicia el planificador Erlang conectado a una de ellas:

1. Compila ambos componentes (`c_agent` y los módulos Erlang) si es necesario.
2. Levanta el Agente C A en el puerto y con los recursos indicados por parámetro.
3. Levanta el Agente C B en un segundo puerto, con sus propios recursos.
4. Espera unos segundos para que ambos agentes se descubran mutuamente vía UDP.
5. Inicia el planificador Erlang, conectado al puerto local del Agente A.

La verificación de que no ocurrió un interbloqueo se realiza inspeccionando el archivo de log (`event_logger.txt`) generado por el planificador: si el sistema funciona correctamente, deben aparecer eventos `job_granted` para los trabajos generados, incluso en el escenario donde dos trabajos compiten por los mismos recursos en el mismo orden que el ejemplo de la Sección 4. La ausencia de eventos `job_granted` sostenida en el tiempo, junto con trabajos que permanecen indefinidamente en estado `pending`, sería indicio de un interbloqueo no resuelto.

7. Estructuras de Datos

En primera instancia, nos hemos abstraído de la estructura del nodo particionándola en tres módulos:

- **Nodos conocidos** (`known_nodes`)
- **Recursos locales** (`local_resources`)
- **Cargas de trabajo propias** (`owned_jobs`)

Cada uno de estos módulos está protegido por su propio lock, el cual es manipulado por el modulo `resource_adapter.c`.

A continuación se detalla en qué consiste cada uno de ellos.

7.1. Nodos conocidos

Como el nombre del módulo `known_nodes` indica, este módulo implementa el directorio de nodos conocidos dentro del clúster distribuido. Su objetivo es mantener actualizada la información de todos los agentes descubiertos mediante mensajes UDP `ANNOUNCE`, permitiendo que el sistema conozca qué nodos están disponibles, qué recursos poseen, y eliminando automáticamente aquellos que dejan de responder.

Estructura principal. La estructura principal del módulo es `elem_known_nodes_t`, que representa un único nodo remoto del clúster. Cada instancia almacena la siguiente información:

- **Dirección IP** (`node_ip`): identifica la máquina remota.
- **Puerto TCP** (`node_port`): puerto donde el agente escucha conexiones.
- **Recursos disponibles** (`avail[RES_NUM]`): arreglo que almacena la cantidad disponible de CPU, RAM y GPU reportada por el último mensaje `ANNOUNCE`.
- **Marca temporal** (`time_stamp`): instante en el que se recibió el último anuncio del nodo, utilizada para detectar nodos inactivos.

El conjunto de estas estructuras se almacena en una tabla hash genérica (`TablaHash`), denominada `known_nodes_t`. La tabla utiliza como clave la combinación (`IP`, `Puerto`), lo que permite distinguir correctamente varios agentes ejecutándose sobre una misma máquina. Para ello se implementan funciones específicas de comparación (`comp_known_node`) y de hash (`hash_known_node`).

Operaciones del módulo. El módulo ofrece operaciones para:

- Crear y destruir el directorio de nodos.
- Actualizar o insertar un nodo cuando se recibe un nuevo mensaje `ANNOUNCE`, reemplazando automáticamente la información anterior y renovando su marca temporal.
- Generar el mensaje `NODES`, recorriendo la tabla hash mediante el patrón *visitor* para construir dinámicamente la lista de nodos y sus recursos disponibles.
- Eliminar nodos inactivos, detectando aquellos cuyo tiempo sin anuncios supera el límite establecido (`ANNOUNCE_TIMEOUT`). Durante este proceso también se devuelve la lista de nodos eliminados para que otros módulos puedan liberar o cancelar los trabajos asociados.

7.2. Recursos locales

El módulo `local_resource` administra los recursos físicos del nodo local y controla el ciclo de vida de las solicitudes de reserva realizadas por los nodos remotos. Su responsabilidad principal es decidir si una petición de recursos puede satisfacerse inmediatamente, debe quedar en espera, o debe ser rechazada, manteniendo en todo momento un registro consistente de los recursos disponibles y de las asignaciones activas.

Estructuras principales. Para ello emplea dos estructuras principales:

- `elem_active_job_t`, que representa una reserva de recursos ya concedida. Cada elemento almacena:

- el identificador del trabajo (`job_id`),
- el socket asociado al nodo remoto (`socket`),
- la cantidad de recursos asignados (`resource_quantity`),
- el tipo de recurso reservado (`resource_type`: CPU, GPU o RAM).

Estas estructuras se almacenan en una tabla hash (`active_jobs_t`), cuya clave está formada por la combinación (`job_id`, `socket`, `resource_type`), permitiendo identificar de manera única cada reserva activa.

- `elem_job_request_t`, que representa una solicitud pendiente de recursos cuando el nodo no dispone de capacidad suficiente para satisfacerla inmediatamente. Contiene la misma información que un trabajo activo (identificador, socket, cantidad y tipo de recurso), pero estas estructuras se almacenan en colas FIFO independientes para cada tipo de recurso, garantizando que las solicitudes se atiendan en orden de llegada.

La estructura `local_resources_t` (definida en el archivo de cabecera) mantiene el estado general del nodo, almacenando:

- la cantidad total de recursos físicos (`total`),
- la cantidad actualmente disponible (`avail`),
- una cola de espera (`request_queue`) para cada tipo de recurso.

Funcionalidades principales. El módulo implementa las siguientes funcionalidades principales:

- Inicializar y destruir la estructura que representa los recursos físicos del nodo.
- Registrar nuevas solicitudes de recursos, concediéndolas inmediatamente cuando existe disponibilidad o encolándolas en caso contrario.
- Liberar recursos cuando un trabajo finaliza, actualizando el inventario del nodo.
- Revisar periódicamente las colas de espera para determinar si alguna solicitud pendiente puede ser satisfecha tras la liberación de recursos.
- Eliminar automáticamente todas las reservas y solicitudes asociadas a un nodo que se haya desconectado, evitando pérdidas permanentes de recursos y situaciones de bloqueo del sistema.

Estructuras de datos empleadas. Desde el punto de vista de las estructuras de datos, el módulo utiliza:

- **Tabla hash**, para almacenar y acceder eficientemente a las reservas activas.
- **Colas FIFO**, para administrar las solicitudes pendientes respetando el orden de llegada.

7.3. Cargas de trabajo propias

El módulo `owned_jobs` administra los trabajos generados por el planificador Erlang local. Su función es mantener un seguimiento completo del estado de cada trabajo distribuido, registrando qué recursos fueron solicitados a cada nodo remoto, cuáles ya fueron concedidos y cuáles continúan pendientes. De esta manera, actúa como el coordinador del ciclo de vida de los trabajos iniciados por el nodo local.

Estructura principal. La estructura principal del módulo es `owned_jobs_t`, implementada como una tabla hash (`TablaHash`) cuyos elementos representan un trabajo distribuido identificado de forma única por su `job_id`. Cada trabajo almacena toda la información necesaria para coordinar las reservas realizadas sobre distintos nodos del clúster.

Para cada trabajo se registran:

- **Identificador del trabajo** (`job_id`), generado por el planificador Erlang.
- **Socket del proceso propietario**, utilizado para identificar al cliente que inició el trabajo.
- **Conjunto de peticiones de recursos**, donde cada petición especifica:
 - la dirección IP del nodo remoto,
 - el puerto TCP del nodo,
 - el tipo de recurso solicitado (CPU, GPU o RAM),
 - la cantidad requerida,
 - el estado de la petición (pendiente o concedida).
- **Información temporal**, utilizada para detectar trabajos que exceden el tiempo máximo de espera (`JOB_TIMEOUT`).
- **Límite de recursos por trabajo** (`MAX_JOB_RESOURCES`), que restringe la cantidad máxima de reservas asociadas a un mismo trabajo.

Operaciones del módulo. El módulo proporciona operaciones para:

- Registrar un nuevo trabajo generado por el planificador local.
- Agregar progresivamente las distintas peticiones de recursos que compondrán el trabajo distribuido.
- Marcar una petición como concedida cuando un nodo remoto responde con un mensaje `GRANTED`, verificando automáticamente si el trabajo ya obtuvo todos los recursos necesarios.
- Obtener la siguiente reserva pendiente para construir mensajes `CMD_RESERVE`.
- Recuperar únicamente los recursos efectivamente concedidos cuando un trabajo debe cancelarse, permitiendo liberar dichos recursos mediante mensajes `CMD_RELEASE`.
- Detectar trabajos afectados por la caída de un nodo remoto, identificando aquellos que dependían de recursos alojados en dicho nodo.
- Detectar trabajos que permanecen demasiado tiempo esperando respuestas de la red y eliminarlos por *timeout*.
- Eliminar definitivamente un trabajo cuando finaliza correctamente o cuando es cancelado.

7.4. Sobre las implementaciones de los TADs empleados

Complejidades del módulo `queues.c`.

Complejidades promedio del módulo `tablahash.c`.

Operación	Complejidad	Descripción
<code>queue_create</code>	$O(1)$	Inicializa una cola vacía devolviendo NULL.
<code>queue_destroy</code>	$O(n)$	Recorre todos los elementos de la cola liberando su memoria.
<code>queue_empty</code>	$O(1)$	Solo verifica si el puntero de la cola es NULL.
<code>queue_add</code>	$O(n)$	Inserta al final de la cola recorriendo todos los nodos hasta encontrar el último.
<code>queue_first</code>	$O(1)$	Accede únicamente al primer elemento de la cola.
<code>queue_remove</code>	$O(1)$	Elimina el primer nodo actualizando el puntero de la cabeza.
<code>queue_delete</code>	$O(n)$	Puede requerir recorrer toda la cola hasta encontrar el elemento buscado.
<code>queue_update</code>	$O(n)$	Busca el elemento a modificar recorriendo secuencialmente la cola.

Cuadro 1: Complejidades de las operaciones del módulo `queues.c`

Operación	Prom.	Peor caso	Descripción
<code>tablahash_crear</code>	$O(m)$	–	Inicializa las m casillas de la tabla.
<code>tablahash_nelems</code>	$O(1)$	–	Devuelve un contador almacenado en la estructura.
<code>tablahash_capacidad</code>	$O(1)$	–	Devuelve el tamaño actual de la tabla.
<code>tablahash_destruir</code>	$O(m)$	–	Recorre todas las posiciones liberando los elementos almacenados.
<code>tablahash_insertar</code>	$O(1)$ amortizado	–	La inserción requiere recorrer el clúster correspondiente; si ocurre una re-dimensión, deben reinsertarse todos los elementos.
<code>tablahash_buscar</code>	$O(1)$	$O(m)$	La búsqueda inspecciona únicamente el clúster asociado al valor hash; en el peor caso puede recorrer toda la tabla.
<code>tablahash_eliminar</code>	$O(1)$	$O(m)$	Localiza el elemento mediante sondeo lineal y marca la casilla como eliminada.
<code>tablahash_visitar</code>	$O(m)$	$O(m)$	Recorre todas las posiciones de la tabla aplicando una función visitante.
<code>tablahash_visitar_extra</code>	$O(m)$	$O(m)$	Igual que la anterior, pero pasando un contexto adicional al visitante.

Cuadro 2: Complejidades promedio de las operaciones del módulo `tablahash.c`

8. Desafíos de Implementación y Soluciones

Durante el desarrollo del gestor de recursos distribuido, se identificaron y resolvieron varios problemas de implementación. Estos problemas influyeron tanto en el protocolo de comunicación como en la arquitectura interna del sistema.

8.1. Identificación de Agentes

Inicialmente, los agentes C se identificaban únicamente por su dirección IP. Este enfoque resultó insuficiente cuando se ejecutaban múltiples agentes en el mismo host, ya que diferentes nodos podían compartir la misma IP mientras escuchaban en puertos distintos. Para resolver este problema, los agentes se identificaron de forma única utilizando el par:

(IP, Puerto)

Este identificador pasó a formar parte del descubrimiento de nodos, la comunicación remota y los mensajes de asignación de recursos, lo que permitió que múltiples agentes coexistieran en la misma máquina.

8.2. Propiedad de Recursos Remotos

A medida que el sistema evolucionó para admitir múltiples agentes, los recursos podían pertenecer a diferentes nodos. Una solicitud de trabajo podría requerir recursos distribuidos en varias máquinas. Por esta razón, los descriptores de recursos se ampliaron para incluir la información del nodo de destino. Por lo tanto, las operaciones de reserva y liberación se dirigen al agente remoto correcto, lo que garantiza que los recursos se asignen y liberen de manera consistente.

8.3. Tráfico de Descubrimiento UDP

El mecanismo de descubrimiento distribuido se basa en anuncios periódicos de UDP entre agentes. Las primeras versiones utilizaban un intervalo de anuncio muy corto, lo que producía un tráfico de red innecesario y una salida de log excesiva. Además, los parámetros de inicio incorrectos ocasionalmente causaban mensajes de anuncio no válidos. El protocolo de descubrimiento se estabilizó aumentando el intervalo de difusión (broadcast) y estandarizando el formato del anuncio:

```
1 ANNOUNCE <port> cpu:<cpu> mem:<memory> gpu:<gpu>
```

Esto redujo el ruido de la red manteniendo un descubrimiento de nodos confiable.

8.4. Colisión de Job ID entre Nodos en la Misma IP

Problema: La asignación específica que dos nodos pueden compartir la misma dirección IP siempre que utilicen puertos diferentes. Dado que el programador (scheduler) de Erlang de cada nodo genera IDs de trabajo de forma independiente utilizando `erlang:unique_integer/1`, dos programadores en la misma IP pueden producir IDs de trabajo idénticos. Cuando un tercer agente recibe un mensaje `RESERVE 42 cpu 2`, no puede determinar qué nodo lo envió basándose únicamente en el ID de trabajo.

Solución: El agente en C ya tiene acceso al descriptor de archivo (`fd`) de la conexión TCP entrante a través del parámetro `SOCKET` en `resource_adapter_patch`. Dado que el kernel garantiza que no hay dos conexiones activas que compartan el mismo `fd` dentro de un proceso, el par (`job_id`, `fd`) es globalmente único:

- El `fd` identifica de forma única qué nodo envió el mensaje.
- Dentro de la misma conexión (`fd`), `erlang:unique_integer` garantiza que el ID de trabajo nunca se repita.

La solución no requirió cambios en el protocolo de red: el `fd` ya está disponible en toda la tubería de manejo de eventos y solo fue necesario incorporarlo como parte de la clave de trabajo en la tabla de trabajos activos.

8.5. Cierre Prematuro del Socket

Problema: Cuando el proceso que abrió un socket TCP finaliza, Erlang cierra el socket automáticamente. Esto provocaba que la conexión se cayera cuando `conn_handler` generaba procesos hijos y luego terminaba.

Solución: El ID del socket se pasa directamente a los procesos `sender` (emisor) y `receiver` (receptor) en el momento de su creación. Dado que estos son procesos de larga duración, el socket permanece abierto durante toda la sesión.

8.6. Registro del ID del Proceso de Registro (Logger)

Problema: Pasar el PID del logger a través de cada llamada a función creaba ruido innecesario entre los módulos.

Solución: El logger se registra bajo el átomo `eventLoggerId` mediante `register/2`. Todos los módulos envían mensajes de registro directamente al átomo.

8.7. Bloqueo de Lectura TCP vs. Paso de Mensajes en Erlang

Problema: Un proceso bloqueado en `gen_tcp:recv` con `{active, false}` no puede recibir mensajes de Erlang simultáneamente. La alternativa, `{active, true}`, entrega los datos TCP directamente al buzón del proceso; pero si el agente C envía mensajes más rápido de lo que el proceso los consume, el buzón puede agotar la memoria.

Solución: Mantuvimos `{active, false}` y separamos el tráfico entrante y saliente en dos procesos dedicados:

- `receiver`: se bloquea en `gen_tcp:recv` esperando las respuestas del agente C.
- `sender`: se bloquea en `receive` esperando las directivas del programador.

Cada proceso tiene una única responsabilidad de bloqueo, evitando ambos problemas.

8.8. Fragmentación de Mensajes TCP (Framing)

Problema: TCP es un protocolo orientado a flujo de bytes, no a mensajes discretos. Una única llamada de lectura puede entregar un mensaje incompleto, o varios mensajes concatenados en un mismo bloque de datos (por ejemplo, la respuesta `NODES ...` seguida inmediatamente de un `JOB_GRANTED ...` si ambos se generaron con poca diferencia de tiempo). Si esto no se maneja explícitamente, el receptor puede intentar interpretar dos comandos como uno solo, provocando errores de parseo o comandos reconocidos como inválidos.

Solución: Se implementó una protección redundante en ambos extremos de la comunicación. Del lado de Erlang, la conexión se abre con la opción `{packet, line}`, la cual delega en la máquina virtual de Erlang la tarea de entregar exactamente una línea completa (delimitada por `\n`) por cada mensaje recibido, sin importar cómo hayan llegado los datos a nivel de socket. Del lado del agente C, al recibir datos se particiona el buffer utilizando `strtok_r` sobre el carácter `\n`, procesando cada línea como un comando independiente antes de continuar con la siguiente.

Esta doble protección asegura que ningún comando se interprete de forma parcial o combinada, independientemente de cómo el sistema operativo fragmente o agrupe los datos en el socket.

9. Contribuciones del Equipo

Miembro	Contribuciones
Lucio Oviedo Dell'Orefice	Agente C: servidor epoll, manejo de TCP, E/S no bloqueante.
Juan Ignacio Gorgo	Agente C: gestión de recursos, colas FIFO, lógica GRANTED/DENIED.
Camila Sarasúa	Erlang: planificador de trabajos, prevención de interbloqueos, máquina de estados de los trabajos.
Santiago Salvático	Erlang: <code>erlang_c_bridge</code> , <code>event_logger</code> , <code>header.hrl</code> , documentación, integración.